

FACOFFEE

Serviço de Usuários

Relatório de Evidências de Testes

Engenharia de Software — FACOM/UFMS

1. Visão Geral

Este documento apresenta as evidências dos testes realizados no serviço de usuários (users-service) da aplicação FACOFFEE, um sistema web para auxiliar na divisão de custos da copa da FACOM/UFMS. O serviço foi implementado seguindo o padrão de microsserviços Decomposition by Business Capability, sendo responsável exclusivamente pelo domínio de usuários, com banco de dados próprio e sem compartilhamento com outros serviços.

Tecnologias utilizadas

Tecnologia	Uso
Node.js + Express	Runtime e framework HTTP
Prisma + SQLite	Banco de dados local
Keycloak	Autenticação e autorização (JWT)
RabbitMQ	Publicação de eventos de domínio
Jest + Supertest	Testes unitários e de integração

2. Endpoints Implementados

Método	Rota	Auth	Descrição
POST	/users	Público	Criar usuário
GET	/users	MANAGER	Listar usuários
GET	/users/:userId	MANAGER ou próprio	Obter por ID
PATCH	/users/:userId	MANAGER ou próprio	Atualizar dados
DELETE	/users/:userId	MANAGER ou próprio	Desativar usuário
PUT	/users/:userId/roles	MANAGER	Substituir roles

3. Testes Manuais — Thunder Client

Os testes manuais foram realizados utilizando o Thunder Client, extensão do VS Code, com a infraestrutura rodando via Docker Compose (Keycloak, RabbitMQ, Nginx) e o serviço users-service na porta 3001.

3.1 Cenário 01 — Criar usuario (POST /users)

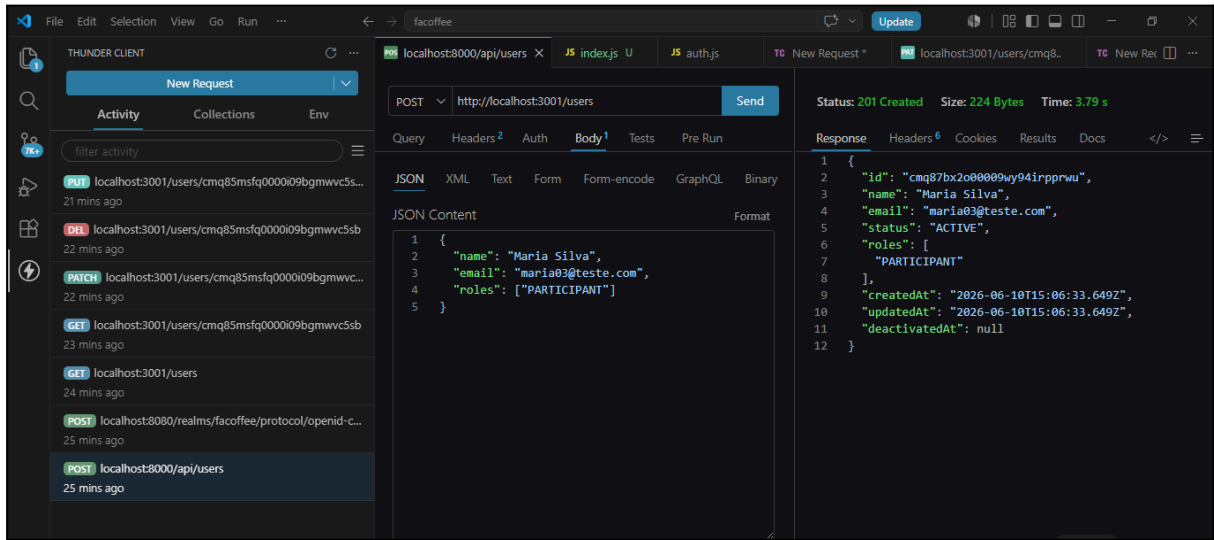
Requisição enviada:

- Método: POST
- URL: `http://localhost:3001/users`
- Body: `{ "name": "Maria Silva", "email": "maria@teste.com", "roles": ["PARTICIPANT"] }`
- Auth: nenhuma (rota publica)

Resultado esperado: 201 Created

Resultado obtido: 201 Created

Campo	Valor retornado
id	cmq85msfq0000i09bgmwvc5sb
name	Maria Silva
email	maria03@teste.com
status	ACTIVE
roles	["PARTICIPANT"]
createdAt	2026-06-10T15:06:33.649Z

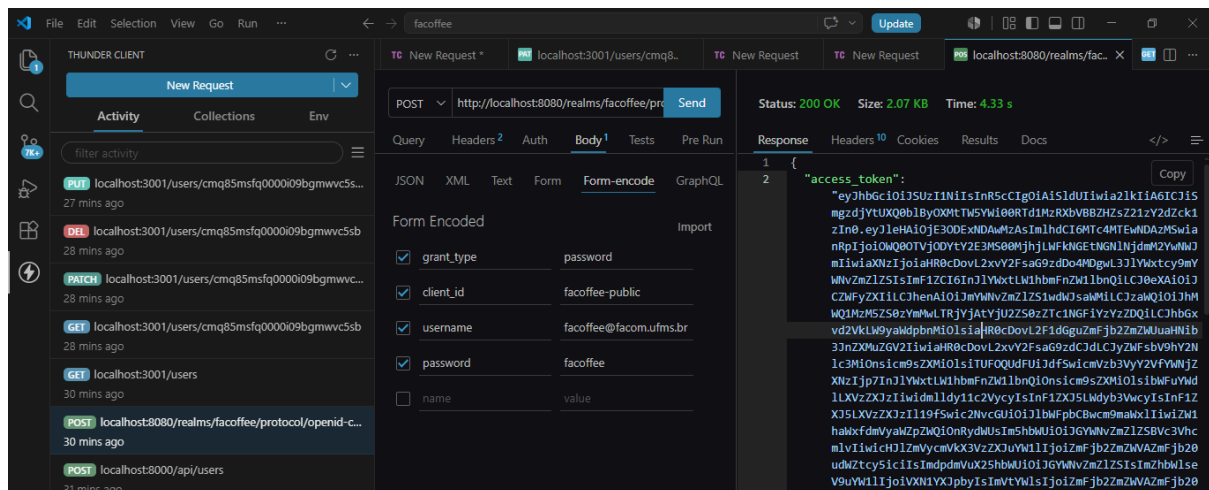


3.2 Cenário 02 — Obter token do Keycloak

Requisição enviada:

- Método: POST
- URL: `http://localhost:8080/realms/facoffee/protocol/openid-connect/token`
- Body (Form Encoded): `grant_type=password, client_id=facoffee-public, username=facoffee@facom.ufms.br, password=facoffee`

Resultado obtido: 200 OK com `access_token` (role: MANAGER)

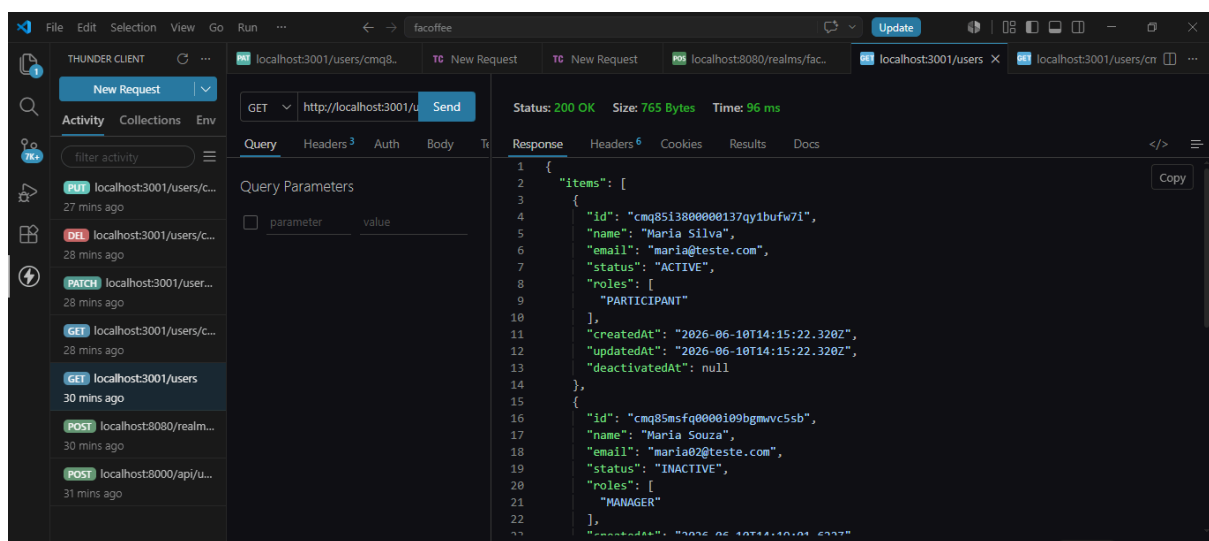


3.3 Cenário 03 — Listar usuários (GET /users)

Requisição enviada:

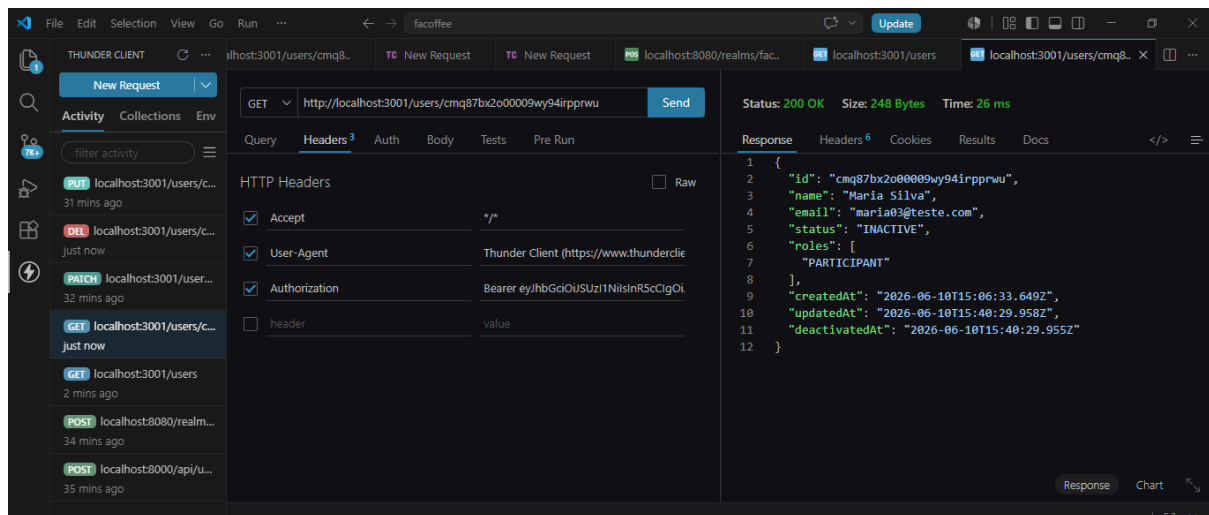
- Método: GET
- URL: `http://localhost:3001/users`
- Header: Authorization: Bearer <token>

Resultado obtido: 200 OK — lista paginada com 2 usuários, `totalElements: 2`, `totalPages: 1`



3.4 Cenário 04 — Buscar usuario por ID (GET /users/:userId)

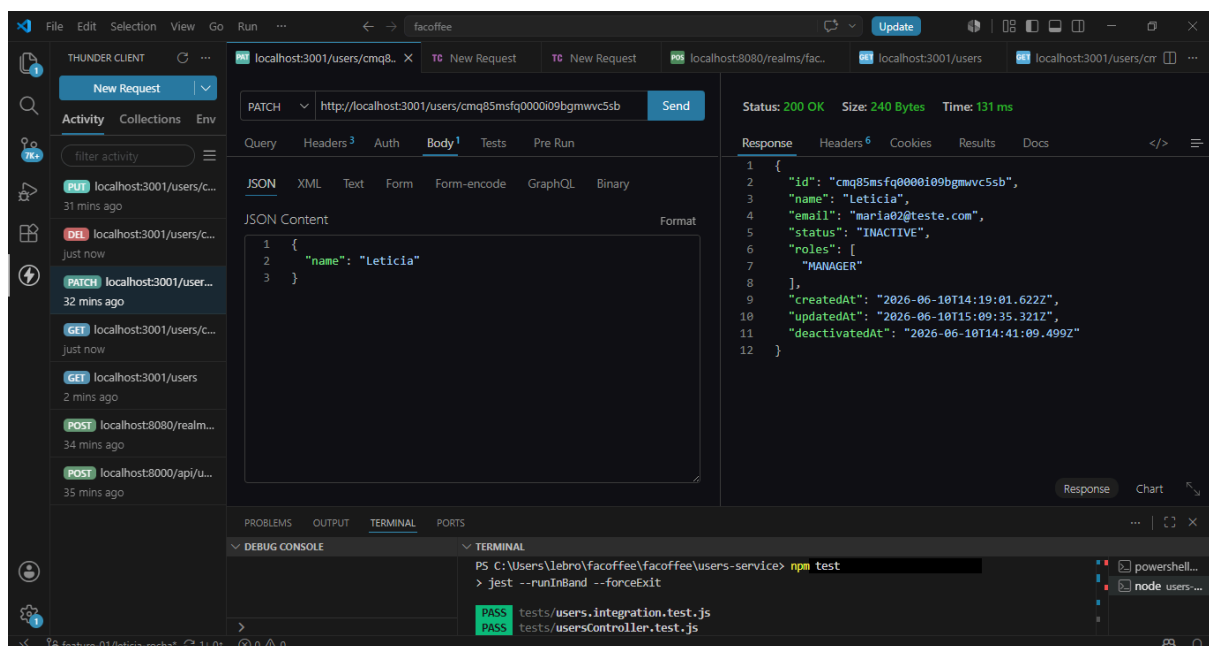
Resultado obtido: 200 OK — dados do usuário retornados corretamente



3.5 Cenário 05 — Atualizar nome (PATCH /users/:userId)

Body enviado: { "name": "Leticia" }

Resultado obtido: 200 OK — campo name atualizado para "Maria Souza", updatedAt atualizado

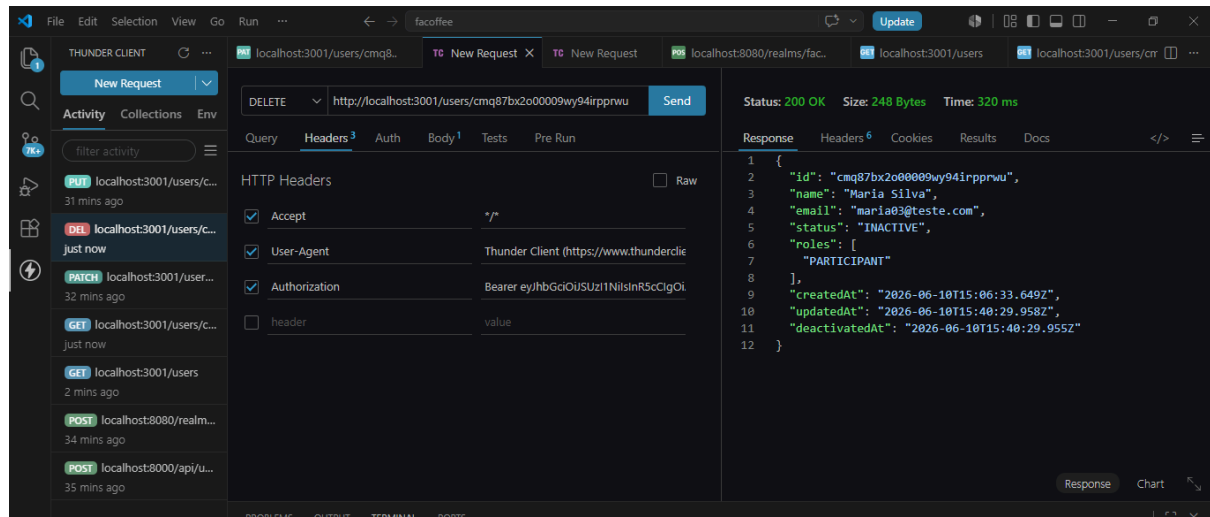


3.6 Cenário 6 — Desativar usuário (DELETE /users/:userId)

Body enviado: { "reason": "Usuário não participa mais da copa" }

Resultado obtido: 200 OK — status alterado para INACTIVE, deactivatedAt preenchido

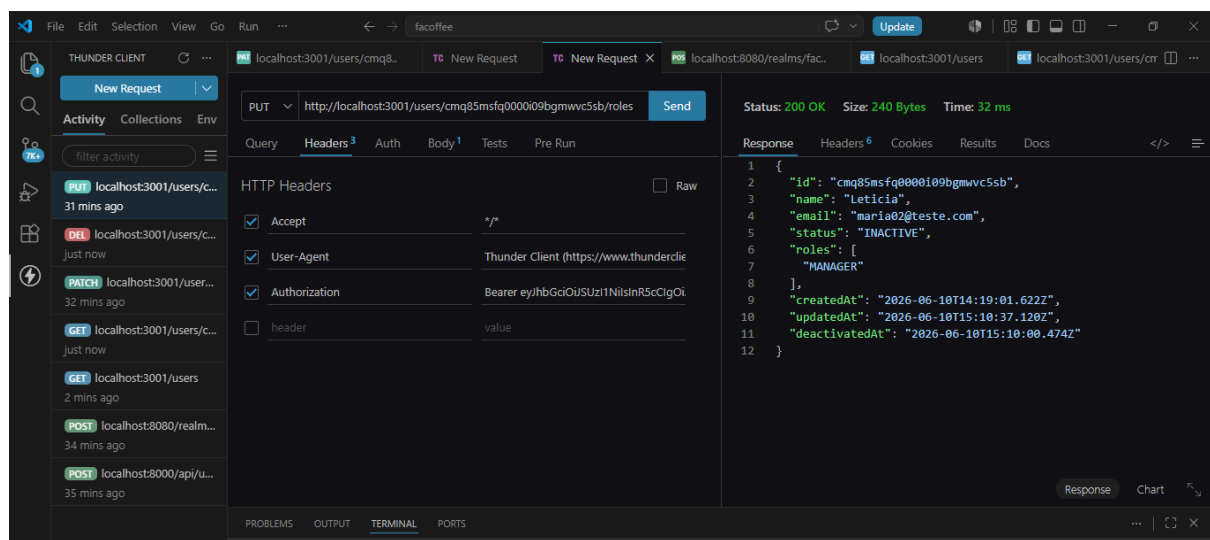
Evento user.deactivated publicado no RabbitMQ — confirmado no terminal: "RabbitMQ conectado"



3.7 Cenário 7 — Substituir roles (PUT /users/:userId/roles)

Body enviado: { "roles": ["MANAGER"] }

Resultado obtido: 200 OK — roles substituídas para ["MANAGER"]



4. Testes Automatizados

Os testes automatizados foram implementados com Jest e Supertest, cobrindo testes unitários das regras de negócio e testes de integração dos endpoints HTTP.

4.1 Resultado da execução

Métrica	Resultado
Test Suites	2 passed, 2 total
Tests	37 passed, 37 total
Tempo de execução	1.45s
Snapshots	0 total

4.2 Testes unitários — usersController.test.js

Caso de teste	Esperado	Obtido
Criar usuário com sucesso	201	201 PASS
Nome com menos de 3 caracteres	400	400 PASS
Email ausente	400	400 PASS
Email já cadastrado (conflito)	409	409 PASS
Role padrão PARTICIPANT	200	200 PASS
Listar usuários paginado	200	200 PASS
Filtrar por status	200	200 PASS
Buscar usuário existente	200	200 PASS
Buscar usuário inexistente	404	404 PASS
Atualizar nome	200	200 PASS
Atualizar usuário inexistente	404	404 PASS
Desativar usuário	200 INACTIVE	200 PASS
Desativar sem motivo	400	400 PASS
Motivo com menos de 3 chars	400	400 PASS
Desativar usuário inexistente	404	404 PASS
Substituir roles	200	200 PASS

Roles vazio	400	400 PASS
Substituir em usuario inexistente	404	404 PASS

4.3 Testes de integração — users.integration.test.js

Caso de teste	Esperado	Obtido
POST — criar usuario	201	201 PASS
POST — nome invalido	400	400 PASS
POST — email ausente	400	400 PASS
POST — conflito de email	409	409 PASS
GET /users — MANAGER autorizado	200	200 PASS
GET /users — PARTICIPANT barrado	403	403 PASS
GET /users — sem token	401	401 PASS
GET /:id — MANAGER acessa qualquer	200	200 PASS
GET /:id — PARTICIPANT acessa proprio	200	200 PASS
GET /:id — PARTICIPANT barrado	403	403 PASS
GET /:id — usuario inexistente	404	404 PASS
PATCH — MANAGER atualiza	200	200 PASS
PATCH — PARTICIPANT barrado	403	403 PASS
DELETE — desativar usuario	200 INACTIVE	200 PASS
DELETE — sem motivo	400	400 PASS
DELETE — PARTICIPANT barrado	403	403 PASS
PUT /roles — MANAGER substitui	200	200 PASS
PUT /roles — roles vazio	400	400 PASS
PUT /roles — PARTICIPANT barrado	403	403 PASS

5. Eventos de Domínio Publicados

Os eventos foram publicados no RabbitMQ durante a execucao dos testes manuais, confirmados pelos logs no terminal do servidor.

Evento	Trigger	Status
user.created	POST /users	Publicado
user.deactivated	DELETE /users/:userId	Publicado
user.roles.replaced	PUT /users/:userId/roles	Publicado

6. Padrão de Microserviços Aplicado

Decomposition by Business Capability

O padrão Decomposition by Business Capability define que cada microservico deve ser responsável por uma única capacidade de negócio, com autonomia total sobre seus dados e lógica. Evidências da aplicação do padrão no users-service:

- Banco de dados próprio (SQLite via Prisma) — não compartilhado com outros serviços
- Domínio de usuários encapsulado exclusivamente neste serviço
- Endpoints, regras de negócio, persistência e eventos próprios
- Comunicação com outros serviços via eventos assíncronos (RabbitMQ) — sem acoplamento direto
- Serviço deployado independentemente na porta 3001

7. Conclusão

O serviço de usuários foi implementado e testado com sucesso, atendendo a todos os criterios de aceite definidos no GUIA_EQUIPE_USERS.md:

- Todos os 6 endpoints implementados e funcionando conforme o contrato api-docs.yaml
- Validação JWT e autorização por roles (MANAGER/PARTICIPANT) implementadas
- Integração com Keycloak para criacao e gestao de usuários
- Eventos de domínio publicados no RabbitMQ
- 37 testes automatizados passando (18 unitários + 19 de integração)
- Banco de dados próprio sem compartilhamento
- Padrão Decomposition by Business Capability aplicado e documentado